

---

# Algorithmique et Programmation 1

Notes de cours Semaine 3

Team AP1

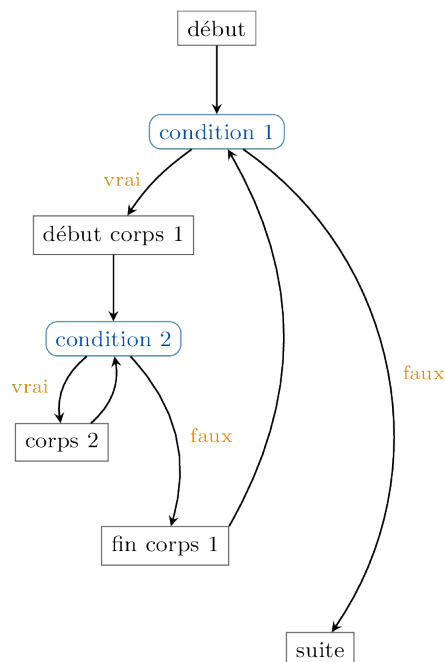
29 septembre 2025

## Table des matières

|                                                          |           |
|----------------------------------------------------------|-----------|
| Boucles imbriquées . . . . .                             | 1         |
| <b>Introduction aux listes</b>                           | <b>5</b>  |
| Création et affichage . . . . .                          | 7         |
| Opérations et fonctions de base . . . . .                | 7         |
| Manipulations par méthodes . . . . .                     | 10        |
| <b>Introduction aux fonctions</b>                        | <b>11</b> |
| Fonctions prédéfinies et bibliothèque standard . . . . . | 12        |
| Définition de fonction . . . . .                         | 12        |
| Appel de fonction . . . . .                              | 13        |
| Exemples . . . . .                                       | 13        |

## Boucles imbriquées

On peut écrire une boucle à l'intérieur d'une autre boucle.



**Figure 1** – *Diagramme d'exécution d'une double boucle*

La syntaxe d'une double boucle `while` :

```

1 # début
2 while condition1:
3     # début corps 1
  
```

```
4     while condition2:
5         # corps 2
6     # fin corps 1
7 # suite
```

### Exemple : table d'addition

**But :** afficher toutes les additions de nombres inférieurs à  $n$

```
1 m = 3
2 n = 11
3
4 a = 0 # hors de la boucle externe !
5 while a < m:
6     b = 0 # dans la boucle externe !
7     while b < n:
8         print(a, '+', b, '=', a+b)
9         b += 1 # dans la boucle interne !
10    a += 1 # dans la boucle externe !
```

### Exercices :

1. afficher toutes les additions  $a + b = c$  pour  $a$  entre 0 et  $n$  et  $b$  entre 0 et  $m$
2. essayer d'écrire le programme en utilisant une seule boucle (*non recommandé en temps normal !*)

### Exemple : compter jusqu'à 59, dix nombres par ligne

```
1 # paramètres
2 limite = 60
3 nb_colonnes = 7
4
5 courant = 0
6 while courant < limite:
7     colonne = 0
8     while colonne < nb_colonnes and courant < limite:
9         print(courant, end = '\t')
10        colonne += 1
11        courant += 1
12    print() # affiche un retour à la ligne
```

**Exercice :** (pas si évident) faire en sorte que les nombres successifs apparaissent sur une même colonne

```
1 # paramètres
2 limite = 37
3 nb_colonnes = 10
```

```

4
5 nb_lignes = limite // nb_colonnes + 1
6 ligne = 0
7 while ligne < nb_lignes:
8     colonne = 0
9     courant = ligne
10    while courant < limite and colonne < nb_colonnes:
11        print(courant, end = '\t')
12        courant += nb_lignes
13        colonne += 1
14    ligne += 1
15    print() # affiche un retour à la ligne

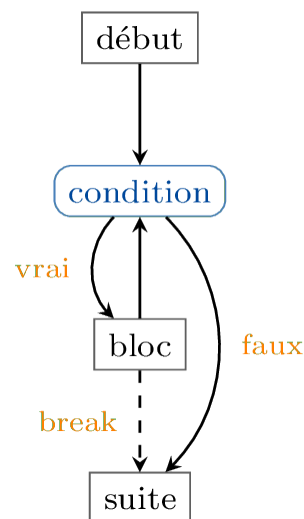
```



### Contrôle de boucle

Tout ce qui suit est *non exigible en contrôle*.

**Instruction break** On peut sortir prématurément d'une boucle `while` - Instruction `break` dans le corps (en général dans une conditionnelle) - Force une sortie de boucle et passe directement à la suite du programme - On ne réévalue pas la condition du `while`



**Figure 2** – Rappel : diagramme d'exécution d'une boucle `while`

**Exemple** : saisie contrôlée sans duplication de l'instruction `input` :

```

1 while True:
2     note_cc1 = float(input('Note du premier contrôle : '))
3     if 0 <= note_cc1 <= 20:

```

```
4         # saisie correcte, on termine la boucle
5         break
6     # saisie incorrecte, on recommence
7     print('Erreur de saisie.')
```

### Instruction continue

On peut passer prématurément à l'itération suivante - Instruction `continue` dans le corps de la boucle - On retourne directement à la condition, et on la réévalue

```
1 while True:
2     saisie = input("Dis-moi quelque chose : ")
3     if saisie == "":
4         print("Tu n'as rien dit !")
5         continue
6     if saisie == "je réfléchis": # elif inutile ici !
7         print("OK, j'attends...")
8         continue
9     if saisie == "stop": # elif inutile ici !
10        print("D'accord...")
11        break
12    if saisie == "arrête": # elif inutile ici !
13        print("C'est bon, j'ai compris !")
14        break
15    # else inutile ici !
16    print("Ta phrase fait", len(saisie), "caractères.")
```

### Remarques sur break et continue

- Ces deux instructions ne sont à utiliser que par des programmeurs confirmés
- Elles compliquent en général la compréhension et l'analyse du code
- On peut toujours s'en sortir sans (en utilisant des `if`), même si ça peut rendre le code moins "élégant"

```
1 saisie = ""
2 while saisie != "stop" and saisie != "arrête":
3     saisie = input("Dis-moi quelque chose : ")
4     if saisie == "":
5         print("Tu n'as rien dit !")
6     elif saisie == "je réfléchis":
7         print("OK, j'attends...")
8     else:
9         print("Ta phrase fait", len(saisie), "caractères.")
10 if saisie == "stop":
11     print("D'accord...")
12 elif saisie == "arrête":
13     print("C'est bon, j'ai compris !")
```

```
1 a, b, c = ...
2
3 # Calcul et affichage du nombre de solutions
4 if a == b == c == 0:
5     print("Une infinité de solutions réelles (et complexes)")
6 elif a == b == 0: # A ce stade, si a = b = 0, c est différent de
    0 !!!
7     print("Aucune solution")
8 elif a == 0 :      # A ce stade, si a = 0, b est différent de 0
    !!!
9     print("Une solution réelle")
10 else:
11     delta = b ** 2 - 4 * a * c
12     if delta < 0:
13         print("Pas de solution réelle")
14     elif delta == 0:
15         print("Une solution réelle")
16     else:
17         print("Deux solutions réelles")
```

## Introduction aux listes

Il arrive souvent que l'on veuille faire référence à plusieurs données en même temps (parce qu'ils sont de même type, ou parce qu'elles se rapportent au même objet, par exemple). Par ailleurs, on ne sait pas toujours quelle quantité de données on va devoir retenir pour le bon déroulé de notre programme ou algorithme. Difficile dans ces cas de savoir le nombre exact de variables à déclarer et initialiser ! On a donc envie de pouvoir regrouper des données en les organisant en *collections ordonnées*, ayant en plus un caractère *dynamique* et *mutable* : on peut ajouter des éléments à notre collection à notre guise, en supprimer, en modifier...

En algorithmique, on parle souvent de *tableaux*, en Python, le type correspondant est **list**, et on parlera sans arrêt de *listes*.

Nous approfondirons tout cela dans la séquence de cours suivante (on verra en particulier en détail les notions de *mutabilité*, d'*itérabilité*, de boucle `* for*`), mais voici déjà quelques bases pour manipuler ces objets un peu spéciaux mais très utiles !

Pour résumer :

**Objectif** : désigner avec une seule variable une collection de valeurs

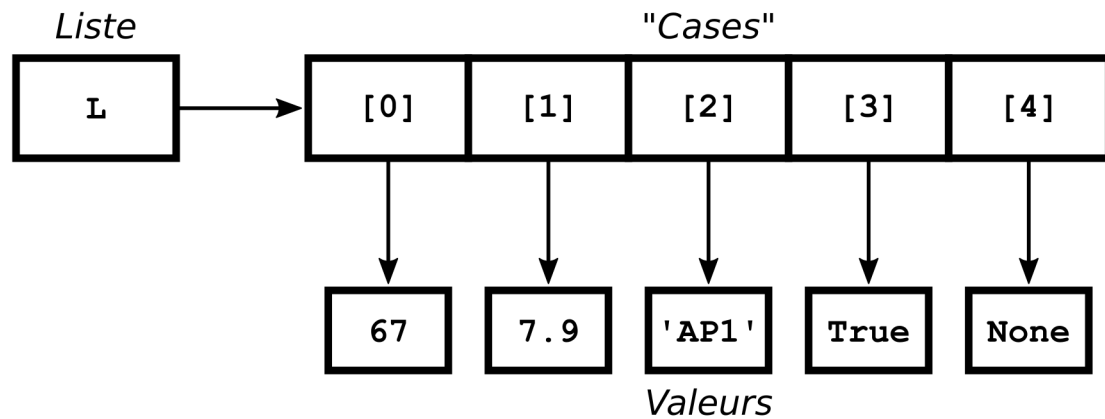
**Liste** : suite **indexée** (numérotée) d'objets quelconques (type `list` en python)

- Éléments "rangés" dans des "cases" numérotées de 0 à  $n - 1$

- En mémoire : tableau à  $n$  cases, chacune contenant une référence (“flèche”) vers un objet
- Peut contenir des objets de plusieurs types différents
- **Mutable** : peut être modifiée, agrandie, raccourcie...



**Figure 3** – *Une métaphore*



### Figure 4 – Un peu plus proche de la réalité

## Création et affichage

**Création :** suite entre [ et ] d'expressions séparées par ,

```
1 >>> lst = [3, 'toto', 4.5, False, None]
2 >>> print(lst)
3 [3, 'toto', 4.5, False, None]
```

Liste vide [] : liste ne contenant aucun objet

```
1 >>> lst = []
2 >>> print(lst)
3 []
```

## Une liste peut contenir d'autres listes !

```
1 >>> lst = ['test', [1, [2], 3]]
2 >>> print(lst)
3 ['test', [1, [2], 3]]
```

Voici un exemple à tester sur [Python Tutor](https://pythontutor.com/visualize.html#code=%23Cr%C3%ce%20que%20lst\_ex%20est%20modifi%C3%A9e%3F%5Cn%22,%20lst\_ex%29%0A%0Alst\_ex%20est%20val\_ex%20est%20modifi%C3%A9e%3F%5Cn%22,%20val\_ex%29%0A%0A%23%0D%0Afrom%20frontend.js&py=3&rawInputLstJSON=%5B%5D&textReferences=false).

## Opérations et fonctions de base

## Longueur d'une liste

La longueur d'une liste (le nombre d'éléments qu'elle contient) s'obtient par la fonction `len`.

```
1 >>> lst = [3, 'toto', 4.5, False, None]
2 >>> print(len(lst))
3 5
```

```
1 >>> print(len([]))
2 0
```

## Accès aux éléments

Les éléments d'une liste à  $n$  éléments sont numérotés de 0 à  $n - 1$ .

Le numéro d'un élément est appelé son *indice*.

L'accès à un élément donné s'appelle l'**indexation**.

```
1 >>> lst = [3, 'toto', 4.5]
2 >>> print(lst[1])
3 toto
```



### Attention !

- Le premier élément d'une liste est l'élément d'indice 0 !
- Si la liste a  $n$  éléments, il n'existe pas d'élément d'indice  $n$  !
- L'accès à un indice supérieur ou égal à la taille de la liste provoque une erreur !

```
1 lst = [3, 'toto', 4.5]
2 >>> print(lst[3])
3 Traceback (most recent call last):
4   File "<stdin>", line 1, in <module>
5   IndexError: list index out of range
```

**Exercice :** Écrire une fonction qui affiche tous les éléments d'une liste (un par ligne)

```
1 def affiche_elements(lst):
2     ...
3
4 lst = [3, 'toto', 4.5]
5 affiche_elements(lst)
```

```
1 def affiche_elements(lst):
2     i = 0
3     while i < len(lst):
4         print(lst[i])
5         i = i + 1
6
7
8 lst = [3, 'toto', 4.5]
9 affiche_elements(lst)
```

## Modification d'un élément

On peut modifier le  $i$ -ème élément de `lst` à l'aide d'une affectation :

```
1 lst = [3, 'toto', 4.5, False, None]
2 >>> print(lst[2])
3 4.5
4 >>> lst[2] = 'titi'
5 >>> print(lst)
6 [3, 'toto', 'titi', False, None]
```

**Attention**, ceci ne crée pas une nouvelle liste mais modifie la liste sur place !

```
1 lst = [3, 'toto', 4.5, False, None]
2 >>> lst_bis = lst
3 >>> lst[2] = 'titi'
4 >>> lst_bis
5 [3, 'toto', 'titi', False, None]
```

## Concaténation et répétition

Comme pour les chaînes de caractères (`str`) on peut utiliser les opérateurs `+` pour fabriquer la concaténation de deux listes et `*` pour répéter une liste.

```
1 >>> [3, 'toto', 4.5] + [False, None]
2 [3, 'toto', 4.5, False, None]
```

```
1 >>> [] + [3, 'toto', 4.5] + []
2 [3, 'toto', 4.5]
```

```
1 >>> 3 * ['a', 'b']
2 ['a', 'b', 'a', 'b', 'a', 'b']
```

```
1 >>> [0] * 13
2 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

On peut utiliser ces opérateurs pour recopier une liste. Comparer :

```
1 lst = [3, 'toto', 4.5]
2 lst2 = lst
3 lst3 = lst + []
4 lst4 = lst * 1
```

## Test d'appartenance

**Exercice** : Écrire une fonction recevant une liste et une valeur, et renvoyant `True` si la valeur apparaît dans la liste (`False` sinon)

```
1 def appartient(lst, val):
2     ...
```

```
1 >>> def appartient(lst, val):
2 ...     i = 0
3 ...     while i < len(lst):
4 ...         if lst[i] == val:
5 ...             return True
6 ...         i += 1
7 ...     return False
8 ...
9 >>> lst = ['Hildegarde', 'Cunégonde', 'Médor']
10 >>>
11 >>> print(appartient(lst, 'Cunégonde'))
12 True
13 >>>
14 >>> if appartient(lst, 'Médor'):
15 ...     print('Bon chien !')
16 Bon chien !
```

**Remarque :** Cette fonctionnalité existe déjà en Python :

- `val in lst` vaut `True` si `val` apparaît dans `lst`, `False` sinon
- Réciproquement, on peut écrire `val not in lst`

```
1 >>> lst = ['Hildegarde', 'Cunégonde', 'Médor']
2 >>> 'Cunégonde' in lst
3 True
```

```
1 >>> lst = ['Hildegarde', 'Cunégonde', 'Médor']
2 >>> 'Rex' not in lst
3 True
```

```
1 >>> if 'Médor' in lst:
2 ...     print('Bon chien !')
3 ...
4 Bon chien !
```

## Manipulations par méthodes

On va maintenant énumérer un certain nombre de méthodes prédéfinies sur les listes, permettant des modifications plus complexes. Pour plus de détails, on pourra consulter la documentation en ligne.

## Agrandir ou rétrécir une liste

Plusieurs instructions ont un effet sur la taille de la liste :

- L'instruction `lst.append(elem)` ajoute l'élément `elem` à la fin de la liste `lst`
- L'instruction `lst.pop()` supprime le dernier élément de `lst` et renvoie sa valeur
- L'instruction `lst.pop(i)` supprime l'élément d'indice `i` de `lst` et renvoie sa valeur

Les fonctions `append` et `pop` sont appelées **méthodes**, ou fonctions s'appliquant à un objet (nous en verrons d'autres dans les cours suivants)

**Attention**, ces instructions ne créent pas une nouvelle liste mais modifient la liste sur place !

**Attention**, ne pas confondre `x = lst[2]` et `x = lst.pop(2)` !

```
1 >>> lst = [3, 'toto', 4.5, False, None]
2 >>> lst_bis = lst
3 >>>
4 >>> lst.append(1)
5 >>> print(lst_bis)
6 [3, 'toto', 4.5, False, None, 1]
7 >>>
8 >>> elem = lst_bis.pop(2)
9 >>> print(elem)
10 4.5
11 >>>
12 >>> print(lst)
13 [3, 'toto', False, None, 1]
```

## Introduction aux fonctions

En programmation, une fonction est : - un morceau de programme - portant en général un **nom** - acceptant zéro, un ou plusieurs **paramètres** - produisant le plus souvent un **résultat**.

Des exceptions existent, mais la forme la plus courante d'une fonction est donc proche de celle d'une fonction mathématique.

L'utilisation de fonctions améliore les aspects suivants du code :

- **Lisibilité** :
  - isoler une partie du programme (par exemple un gros calcul compliqué)
  - éviter une trop grande imbrication des `if`, des `while`
- **Modularité et robustesse** :
  - réutiliser le même code plusieurs fois (évite de recopier le code)

- faciliter la correction des bugs, l'évolution et la maintenance
- **Généricité** :
  - changer la valeur des paramètres (même calcul mais avec différentes valeurs de départ)

## Fonctions prédéfinies et bibliothèque standard

En Python, il existe un grand nombre de fonctions prédéfinies, que nous avons déjà utilisées, par exemple :

- `int(obj)` : 1 paramètre, 1 résultat. Reçoit en paramètre un objet (par exemple `str` ou `float`), essaie de le transformer en entier et renvoie l'entier obtenu.

```
1 >>> int("34")
2 34
```

- `len(obj)` : 1 paramètre, 1 résultat. Reçoit un objet (par exemple `str`) et renvoie sa longueur.

```
1 >>> len("bonjour")
2 7
```

- `randint(mini, maxi)` : 2 paramètres, 1 résultat. Reçoit deux nombres, et renvoie un entier aléatoire compris entre ces deux nombres (inclus).

```
1 from random import randint
2 randint(1, 34)
```

Il y en a beaucoup d'autres, comme `print`, `input`, `float`, `str`....

Ces fonctions sont appelées *prédéfinies* (ou *built-in*) - il est inutile de les connaître toutes par cœur - liste des fonctions prédéfinies sur cette page

Il existe également de nombreux *modules* officiels (par exemple le module `random`) - bibliothèques de fonctions, de types et d'objets - liste des modules prédéfinis documentée ici.

## Définition de fonction

Pour définir une nouvelle fonction on utilise la syntaxe suivante :

```
1 # ligne suivante : en-tête de fonction
2 def nom_fonction(param_1, ..., param_n):
3     # corps de la fonction
4     # utilisant param_1 à param_n
5     ...
```

```
6     # peut renvoyer un résultat :
7     return resultat
```

Une fonction peut :

- prendre un certain nombre de paramètres (ici,  $n$ , qui s'appellent `param_1` à `param_n`)
- renvoyer une valeur (via l'instruction `return`)

## Appel de fonction

Une fois définie, `nom_fonction` peut être utilisée dans le code (on parle d'un **appel**) en indiquant entre parenthèses ses paramètres séparés par des virgules :

```
1 # définition de fonction
2 def nom_fonction(param_1, ..., param_n):
3     ...
4
5 # reste du programme
6 ...
7 # appel de la fonction :
8 une_var = nom_fonction(expr_1, ..., expr_n)
```

## Exemples

### Fonction à paramètres et résultat

```
1 # fonction à deux paramètres produisant un résultat
2 def maximum(a, b):
3     if a >= b:
4         return a
5     else:
6         return b
```

```
1 nb1 = 14
2 nb2 = 31
3
4 maximum(nb1, nb2) # ne sert à rien !!
5
6 # On appelle la fonction et on garde le résultat dans c :
7 c = maximum(nb1, nb2)
8 print("le max de", nb1, "et", nb2, "est", c)
9
10 # On peut aussi utiliser directement le résultat :
11 print("le max de", nb1, "et", nb2, "est", maximum(nb1, nb2))
```

**Entraînement :**

- Décrire l'exécution pas à pas du programme (avec état de la mémoire). On peut aussi essayer avec Python Tutor.
- Dresser un tableau de valeurs de l'exécution du programme

**Fonction sans paramètre**

- En principe, une fonction sans paramètre devrait avoir toujours le même comportement
- Dans l'exemple suivant, on utilise un générateur pseudo-aléatoire, ce qui explique que la fonction ne renvoie pas toujours le même résultat
- Une fonction pourrait aussi recevoir des données depuis l'extérieur (utilisateur, requête réseau...).

Dans ces cas, on parle de **causes secondaires**

```
1 from random import randint
2
3 def lance_de() :
4     return randint(1,6)
5
6 compteur = 1
7 while lance_de() != 6:
8     compteur = compteur + 1
9 print('Obtenu un 6 en', compteur, 'jets de dé.')
```

**Fonction sans valeur de retour**

- Si l'exécution arrive à la dernière instruction du corps de la fonction sans rencontrer d'instruction `return expr`, alors la valeur de retour par défaut est `None` (même comportement si `return` seul)
- En général une telle fonction a quand même un effet sur l'environnement (affichages, dessin, écriture dans un fichier, envoi d'informations sur le réseau...)

Pour tous les effets autres que le renvoi d'un résultat, on parle d'**effets secondaires**

```
1 from turtle import *
2
3 def trace_polygone(nb_cotes, taille_cote):
4     down()
5     i = 0
6     while i < nb_cotes:
7         forward(taille_cote)
8         left(360 / nb_cotes)
9         i = i + 1
```

```
10
11 # faire une affectation ici ne servirait à rien (essayer !)
12 trace_polygone(5, 100)
13 exitonclick()
```

### Erreur fréquente : confusion (paramètre / saisie) et (retour / affichage)

Les programmeurs débutants confondent très souvent la notion de paramètre et celle de saisie (au clavier par exemple) et la notion de valeur de retour avec celle de valeur affichée.

```
1 # ATTENTION CECI EST INCORRECT, A NE PAS REPRODUIRE !!!
2 def pgcd(a, b):
3     a = int(input()) # NON !
4     b = int(input()) # NON !
5     while a % b != 0:
6         r = a % b
7         a = b
8         b = r
9     print("le pgcd est", b) # NON !
```

### NE SURTOUT PAS PROGRAMMER COMME ÇA !

- Comment écrire un programme vérifiant si trois entiers sont premiers entre eux à l'aide de cette fonction ?
- Combien ce programme ferait-il de saisies ?
- Qu'afficherait ce programme ?